



MORROW

Terminal Product

Complete User & Architecture Manual

Private Intelligence, Built Around You

Version 0.1.0-beta.18 — July 03, 2026

Axiom International · Kitchener-Waterloo, Ontario · getaxiom.ca

Aidan Magee — Owner & Lead Supervisor · aidan@getaxiom.ca

Table of Contents

1. Introduction & Why Morrow

A new class of terminal agent — design philosophy, target audience, what makes it different from Hermes, Claude Code, Codex, and Copilot.

2. Quick Start

Installation on Windows, first launch, onboarding, selecting a project.

3. The Terminal Shell

Premium chrome: MORROW branding, project awareness, mode glyphs, git state, context meter, composer area.

4. Activity & Transparency

Grouped activity, hidden internal noise, progress stage banners, relative paths, /output drill-down.

5. Slash Commands

Complete reference: /help, /mode, /yolo, /model, /project, /tree, /result, /context, /diff, /undo, /output, /panic, /status, /memory, /permissions, /resume, /exit.

6. Permissions & YOLO Mode

Proportional approvals, risk indicators, trust levels (once / session / project), YOLO hard blocks.

7. Autonomous Workflow & Recovery

Plan → Edit → Verify lifecycle, failure recovery, Ctrl+C cancel, /continue resume, recovery hints.

8. Mission Control

Ctrl+T cockpit: Task Tree, Live State, Mission Result tabs with arrow-key navigation.

9. Architecture

Event-driven pipeline: InputState → reduce() → TerminalState → composeApp() → Renderer. Pure, testable, snapshot-friendly.

10. Efficiency Design

Minimizing model calls, duplicate reads, approvals, and context waste. Metrics and measurement.

11. Comparison: Morrow vs. Hermes Agent

Side-by-side analysis of interaction quality, visual hierarchy, project awareness, permissions, and recovery.

12. Installation & Build

From source, pnpm, turbo, Windows/PowerShell requirements.

13. Consumer Acceptance Test

12-step journey through a broken browser-game repair.

A. Appendix: Keyboard Shortcuts

Quick reference card.

1. Introduction & Why Morrow

Morrow is a self-hosted, deeply customizable personal AI agent built by Axiom International. It lives in your terminal — not a browser tab, not a chat window, not a SaaS dashboard. It runs on your machine, against your files, powered by the model and provider you choose. No vendor lock-in. No telemetry. No cloud dependency unless you want it.

Morrow was designed from first principles to be the most polished, understandable, and efficient terminal AI agent in existence. It does not copy Hermes Agent, Claude Code, or Codex — it studies them, learns from their interaction patterns, and builds an original experience that exceeds every one of them in clarity, efficiency, and overall fit-and-finish.

What makes Morrow different

- Every unnecessary line is noise. Every raw tool ID is a bug. Every repeated scan is a bug. Every dead command is a bug. Every unnecessary approval is friction.
- The active project must always be unmistakable. You always know what Morrow is touching, what mode you're in, and how to recover.
- Automatic discovery can skip noisy files, but explicit user requests must still be able to access them.
- Routine work inside the selected project flows without constant interruption. YOLO mode performs normal project work automatically while still confirming destructive, external, privileged, credential-related, or irreversible actions.

Com

Dimension	Morrow	Hermes Agent
Visual hierarchy	MORROW brand, 3-line chrome, project overlays	Verbose header, tool IDs visible
Activity	Grouped, collapsed, stage banners	Individual tool cards, all IDs shown
Commands	18 native TUI commands, zero redirects	Many redirect to line mode
Permissions	3 trust levels, risk indicators, diff previews	Y/N approval only
Recovery	Inline recovery menu with 5 options	Error message only
Mission Control	3-tab cockpit, arrow-key nav	Task tree only
Philosophy	Finish the job. Commercial quality.	Open-source, community-driven

2. Quick Start

Morrow runs on Windows 10/11 x64. Install with a single PowerShell command:

```
irm https://getaxiom.ca/morrow/install | iex
```

After installation, open PowerShell 5.1 or Windows Terminal and run:

```
morrow
```

On first launch, Morrow guides you through onboarding: provider selection, model choice, and privacy preference. Once complete, you land in the interactive terminal shell.

To work on a specific project, navigate to its directory and run `morrow`. Morrow auto-detects the repository root and Git state. You can also select a project explicitly:

```
morrow projects select my-project
```

For a one-shot query without entering the shell:

```
morrow ask "Explain this codebase"
```

For autonomous work with auto-approved edits:

```
morrow yolo "Fix all TypeScript errors"
```

3. The Terminal Shell

The default morrow command opens a polished interactive terminal shell. The shell uses the alternate screen buffer — it paints the entire terminal window as a single frame, updating at a capped frame rate. Cursor is hidden during work, restored on exit. Crash recovery is guaranteed: the exit handler always restores the terminal.

Persistent Header

Three lines of chrome sit at the top of every frame:

- Line 1: MORROW brand (bold), project name (cyan), workspace path (gray)
- Line 2: Mode glyph + label · provider/model · git branch + dirty state · memory · context %
- Line 3: Status word (working/ready/failed) · elapsed time · agent count · process count

Mode
Agent

Conversation Area

User messages are green (you >). Morrow responses are magenta (morrow >). Markdown is rendered inline. Code blocks, plans, warnings, and results appear in the transcript tail. Raw tool IDs, JSON blobs, internal routing, stack traces, and model planning monologues are NEVER exposed in the conversation. All detail is accessible through /output.

Composer

A visual separator bar sits above the persistent bottom input. Multiline paste is supported. Slash-command completion activates on '/'. History recall with ↑↓. Input remains usable while work is running where practical.

Status Bar

The footer shows context-sensitive hints: slash commands · palette shortcut · mission shortcut · history · output viewer · help · exit. During overlays (palette, output, mission), the hint changes to describe the current interaction mode.

4. Activity & Transparency

Morrow replaces debug logs with meaningful grouped activity. Successful tool calls are collapsed. Repeated reads and searches are merged. Internal IDs are hidden by default. Optional missing files are treated calmly. Red is reserved for real failures.

Activity Grouping

Consecutive operations of the same stage are merged into a single compact line:

- 3 file reads → "◦ reading 3 files · package.json, tsconfig.json, src/index.ts"
- 12 search results → "◦ searching 12 results · *.ts"
- Reading + searching + inspecting all merge into "Understanding project" stage

Between the header and conversation, Morrow shows the current stage:

- "Understanding project..." — reading, searching, inspecting
- "Planning changes..." — model is reasoning about approach
- "Editing..." — patches being applied
- "Running checks..." — build, test, lint
- "Verifying..." — post-edit verification
- "Completed" or "Failed" — final status

Path Presentation

Absolute paths under the workspace are shortened to relative. External paths are truncated to their basename. Long paths never dominate the screen.

5. Slash Commands — Complete Reference

All commands execute natively inside the TUI. There are no redirects to line mode.

/help	Show all available commands
/mode [agent inspect plan]	Switch mode. inspect=read-only, plan=plan-only
/yolo [on off status] [policy]	Toggle autonomous approval. Shows hard-block list with 'policy'
/model [id]	Show or change the active model
/project	Display project name, workspace path, branch, and mode
/tree, /tasks	Open task tree view for the current mission
/result	Show mission result aggregate (verification, approvals, integrations)
/context	Show token usage, compaction stats, and trim details
/diff	View the diff of the last task's changes
/undo	Roll back the last task's file changes
/output [tool-id]	View captured output from a tool call or the most recent command
/panic	Emergency stop: disable YOLO and cancel active task
/status	Show current session state summary
/memory	Toggle project context memory on/off
/permissions	Inline viewer showing YOLO state, workspace, and mode glyph legend
/resume, /continue	Resume the most recent paused/interrupted task
/exit, /quit	Exit the shell

Additional: /search <query>, /memory-search <query>, /ps (process list), /checkpoint, /stash <name>, /fork, /skill:<id>, /shortcuts, /theme, /share, /audit, /cost, /bench, /bugs, /versions, /connect, /clear.

6. Permissions & YOLO Mode

Morrow protects users from accidents without blocking legitimate work. The permissions system has three trust levels and a YOLO mode for autonomous operation.

Approval Panel

When a command or patch needs approval, Morrow shows an in-frame panel with:

- The exact command or patch being proposed
- Working directory
- Purpose and risk level (low/medium/high — with color coding)
- Preview of command output or diff changes
- YOLO status reminder with hard-block summary
- Four trust options: [y] approve once, [s] trust session, [p] trust project, [n] deny

YOLO

YOLO (--yolo or /yolo on) enables project-scoped autonomy. Routine work flows without interruption. Morrow still confirms clearly destructive, external, privileged, credential-related, or irreversible actions.

- Auto-approved: file reads/writes, project-scoped commands, patches, searches
- Hard-blocked: secret access, workspace escape, privilege escalation, destructive disk actions, destructive Git, force push, unauthorized network transmission

7. Autonomous Workflow & Recovery

Morrow behaves like a strong senior engineer: it understands unfamiliar repositories, plans concisely, edits accurately, runs builds and tests, recovers from failures, and revises its approach when checks fail.

Workflow

- Understanding — reads manifests, searches for relevant symbols
- Planning — produces a concise plan (shown in-frame)
- Editing — applies targeted patches with diff previews
- Running checks — executes build, test, lint
- Verifying — confirms the fix works before claiming success

Recovery

When a task fails, is interrupted, stalls, or reaches its budget, Morrow shows an inline Recovery section with five next-step commands:

- `/continue` — resume the paused task
- `/diff` — inspect the changes that were made
- `/undo` — roll back all changes
- `/result` — view the full mission result aggregate
- `/output` — see captured tool outputs

Ctrl+
correc

8. Mission Control (Ctrl+T)

Ctrl+T opens Mission Control — a tabbed operational cockpit with three views:

Tab 1: Task Tree

Hierarchical view of the current mission's task tree. Each node shows the task ID (short), status, and kind. Child tasks are indented with tree-drawing characters.

Tab 2: Live State

Real-time summary of: session status, activity count, tool calls, patches, git branch/dirty state, context tokens/percentage, agent count, process count, plan progress.

Tab 3: Mission Result

Full aggregate from the orchestrator: status, provider/model, mode/privacy, plan steps, files affected, commands run, tool calls (with failure count), verification status, approvals, context budget, integrations/conflicts, and next-action guidance.

Navigate with ← → arrow keys or 1/2/3 number keys. Esc or Ctrl+C closes the cockpit.

9. Architecture

Morrow's terminal runtime follows a strict event-driven pipeline. Every producer — the agent runtime, commands, tools, approvals, plans, patches, model streaming — emits a `TerminalEvent`. Producers describe what happened; they never format output. The reducer folds events into `TerminalState`, and the renderer owns everything visible. There is no path from a producer to the screen that bypasses an event.

This decoupling is the core architectural rule. It means the entire UI is snapshot-testable: feed events in, assert lines out, no TTY required.

Pipeline

- User Input → `InputState` reducer (`reduceKey`) → submit / slash / abort actions
- Orchestrator SSE → Task Event Adapter (`mapTaskEvent`) → `TerminalEvent`
- `TerminalState` reducer (`reduce`) ← both event streams fold into unified state
- Activity Grouper (`groupActivities`) collapses consecutive same-stage actions
- View Composer (`composeApp`) → pure lines[] + cursor{row,col}
- Interactive Renderer paints frame with FPS budget, alternate screen, cursor control

Key

<code>apps/cli/src/terminal/session.ts</code>	<code>InteractiveSession</code> — main event loop
<code>apps/cli/src/terminal/state.ts</code>	<code>TerminalState</code> + <code>reduce()</code> reducer
<code>apps/cli/src/terminal/events.ts</code>	<code>TerminalEvent</code> union type + activity kinds
<code>apps/cli/src/terminal/view.ts</code>	Pure views: <code>headerLines</code> , <code>toolCardLines</code> , <code>groupActivities</code> , etc.
<code>apps/cli/src/terminal/app-view.ts</code>	<code>composeApp</code> — full frame with chrome, conversation, input
<code>apps/cli/src/terminal/runtime.ts</code>	<code>InteractiveRenderer</code> — alternate screen, FPS, resizes
<code>apps/cli/src/terminal/input-state.ts</code>	<code>InputState</code> machine — <code>reduceKey</code> , overlays
<code>apps/cli/src/terminal/task-event-adapter.ts</code>	SSE → <code>TerminalEvent</code> mapping
<code>apps/cli/src/terminal/line-renderer.ts</code>	Non-interactive append-only renderer
<code>apps/cli/src/terminal/mission-control.ts</code>	Task tree, result, cockpit formatters
<code>apps/cli/src/terminal/yolo.ts</code>	YOLO policy text, risk indicators
<code>apps/cli/src/terminal/commands.ts</code>	Slash command registry

10. Efficiency Design

Morrow targets DeepSeek-level efficiency. For every task, it minimizes model calls, tool calls, files inspected, repeated reads, prompt bytes, approvals, and retries.

The preferred workflow: reuse cached project summary → use symbol index → read only essential manifests → search narrowly → read relevant files → make targeted edits → verify → summarize.

Metrics Tracked

- Tool-call count and duplicate calls prevented
- Files inspected and context used
- Approvals requested and commands run
- Retries and verification attempts
- Time to first useful action

These
ineffici

11. Comparison: Morrow vs. Hermes Agent

Hermes Agent by Nous Research is the quality benchmark. Morrow does not copy it — it studies its strongest interaction patterns and builds an original experience that exceeds it in visual hierarchy, project awareness, progress clarity, tool presentation, efficiency, permissions, recovery, and autonomy.

Dimension	Morrow	Hermes Agent
Header	3-line chrome, project in cyan, mode glyph in white	Multi-line with labels, tool IDs visible
Activity	Grouped by stage, collapsed, single-line summaries	Individual tool cards with IDs, verbose
Tool display	No IDs, relative paths, [output] links	Raw IDs, absolute paths, full summaries
Approvals	3 trust levels, risk indicators, diff preview	YN/O/D optional trust, basic
Commands	All 18 native in TUI, zero redirects	Many redirect to line mode (MORROW_TUI=0)
Recovery	Inline menu with 5 recovery options	Error message, suggest new session
Mission Ctrl	3-tab cockpit, arrow keys, live state	Task tree only, read from /tree
Slash /help	Compact command list in notice	Multi-line paged output
Exit	Ctrl+C cancel → Ctrl+C confirm exit, terraform Ctrl+C	Ctrl+C exits, terminal may leave artifacts
Efficiency	Groups reads, caches summaries, tracks history	Individual tool calls, no grouping

Morrow's philosophical difference: Hermes is an open-source framework designed for maximum extensibility and community contribution. Morrow is a commercial product designed for maximum clarity, efficiency, and polish in the hands of a professional user. Both are valid. Morrow targets the user who wants a finished product, not a toolkit.

12. Installation & Build

Requirements

- Windows 10/11 x64 (primary target). Linux via source build. macOS planned.
- Node.js ≥ 22 , pnpm ≥ 9 , PowerShell 5.1 or Windows Terminal
- 4+ GB RAM recommended

```
irm https://getaxiom.ca/morrow/install | iex
```

Build from source

```
git clone https://github.com/Mageester/morrow.git
cd morrow
git checkout product/hermes-parity
pnpm install
pnpm build
```

Run from source

```
cd apps/cli
npx tsx src/main.ts
```

The installed CLI is at `bin/morrow.mjs`. Dist artifacts are produced by pnpm build into `dist/` directories per package. The orchestrator service auto-starts on first use. SQLite database at `%LOCALAPPDATA%/morrow/state.db` contains sessions, projects, and task history.

13. Consumer Acceptance Test

To validate the Morrow terminal experience, run through this 12-step journey from a fresh PowerShell 5.1 window:

1. Run `morrow`. Complete onboarding if first launch.
2. Select the broken-game project (`fixtures/broken-game`).
3. Ask Morrow: "Explain this project and identify the bugs."
4. Ask Morrow: "Repair the collision detection and ground offset bugs."
5. Review the activity — observe grouped stages, no raw IDs.
6. Review any approval prompts — note risk indicators, trust options.
7. Use `/context`, `/tree`, `/diff`, `/output`, `/result`, and `/undo`.
8. Open Mission Control with `Ctrl+T`. Navigate all three tabs.
9. Resume the session with `/continue` if it was interrupted.
10. Run the repaired project in a browser.
11. Exit with `Ctrl+C` (twice). Verify terminal restores correctly.
12. Reopen PowerShell and run `morrow resume`. Verify session persistence.

Compare the same workflow against Hermes Agent. Morrow must require fewer or equal tool calls, less context, less user effort, and provide equal or better results with clearer presentation.

Appendix A: Keyboard Shortcuts

All shortcuts work in the interactive terminal shell. Ctrl+C is a multi-stage escape: first press cancels active work; second press (when idle) confirms exit.

Ctrl+C	Cancel active work (first press). Exit shell (second press, idle). Close modals.
Ctrl+T	Open Mission Control (3-tab cockpit)
Ctrl+K	Open command palette
Ctrl+L	Force full-screen repaint
Ctrl+R	Search command history
Ctrl+O	Open /output viewer
?	Show help on empty buffer
↑/↓	Recall command history
Tab	Complete slash command
Esc	Close overlay (palette, output, mission, history)
Ctrl+Enter	Insert newline (multiline input) — Windows
←/→ or 1/2/3	Navigate Mission Control tabs
[y]/[s]/[p]/[n]	Approval: approve once / trust session / trust project / deny

Built by Axiom International

Private Intelligence, Built Around You

Kitchener-Waterloo, Ontario · getaxiom.ca

aidan@getaxiom.ca · 226-753-1833

